

LAB 4

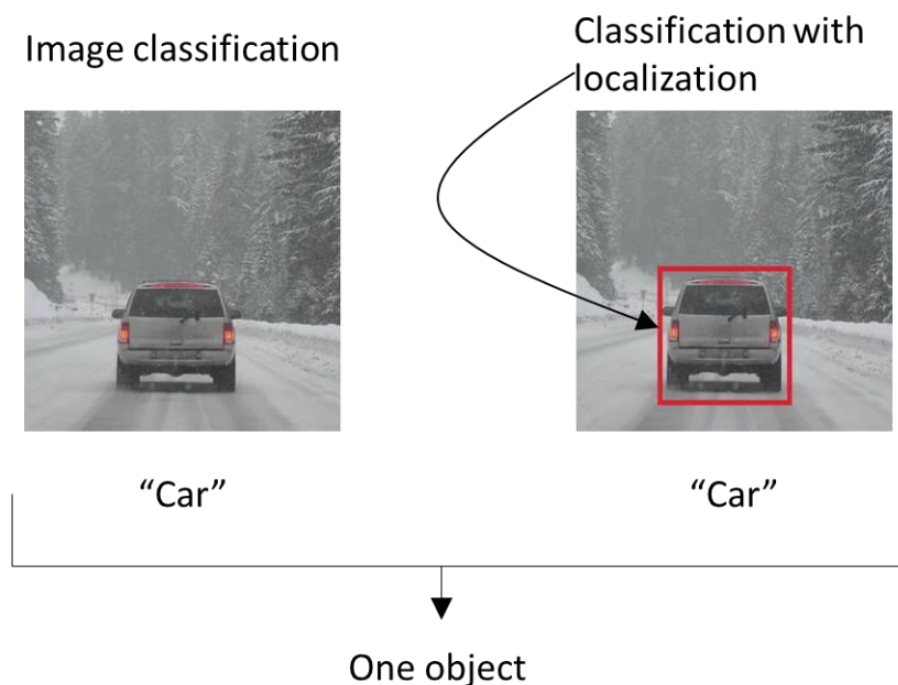
I. Object Localization

Problem

Object Localization requires a model to determine both the position and the label of an object in an image.

The input is an RGB image.

The output consists of two parts: a **class label** and a **bounding box** represented as (cx, cy, w, h) - the center coordinates, width, and height.

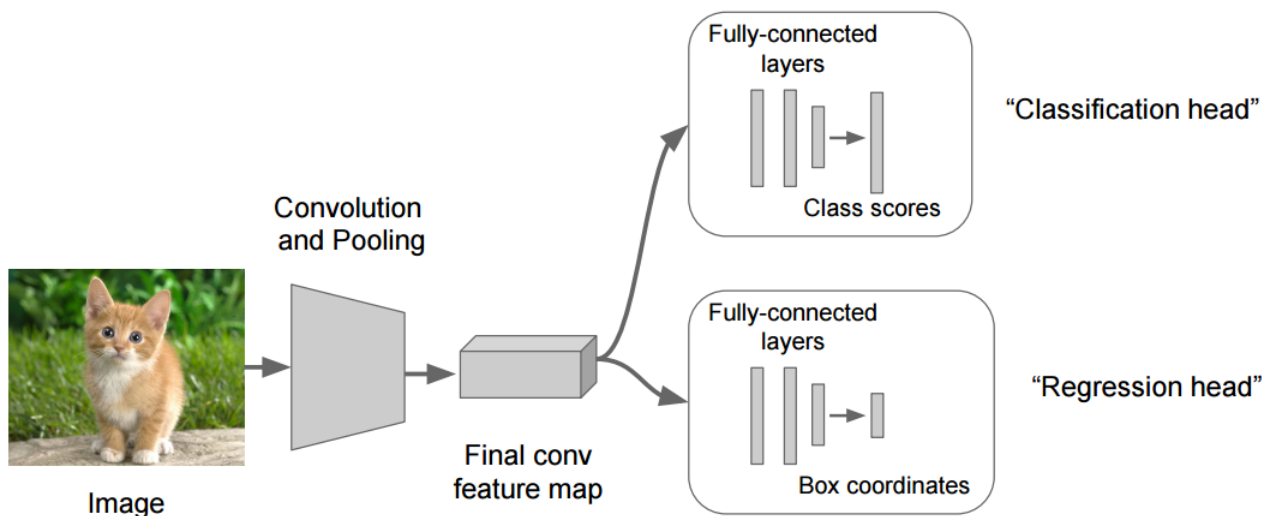


Idea

Localization needs to solve two sub-problems simultaneously: **classification** ("what is this object?") and **coordinate regression** ("where is this object?"). These two tasks are not independent - the features that help recognize *what* the object is also contain information about its shape and location.

Therefore, instead of training two separate models, we use a shared backbone to extract features, then split into two separate branches for each task. This is known as **multi-task learning**, where a shared backbone learns representations for both tasks. This approach is more computationally efficient and often yields better performance because gradients from both tasks jointly update the backbone.

After the backbone extracts a feature vector, the model splits into two heads:



Classification head learns to answer “what is this object?”. It takes the feature vector, passes it through linear layers, and outputs probabilities over classes. The loss used is **Cross-Entropy**, which increases the probability of the correct class while decreasing others.

Regression head learns to answer “where is this object?”. Using the same feature vector, this branch outputs 4 values (cx , cy , w , h). The loss used is **Smooth L1**, which measures the difference between the predicted box and the ground truth.

Because both heads share the same features, gradients from both losses are backpropagated to the backbone. As a result, the backbone is not optimized solely for classification or localization, but is constrained to learn features that are useful for both tasks simultaneously. This is the core advantage of multi-task learning in this setting.

Main implementation code for the above idea: (File *lab-4-localization.ipynb*)

```
class LocalizationNet(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, pretrained=True):
        super().__init__()
        backbone = models.resnet50(weights="IMAGENET1K_V1" if pretrained else None)
        self.features = nn.Sequential(*list(backbone.children())[:-1]) # → (B, 2048, 1, 1)
        feat_dim = 2048

        self.cls_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feat_dim, 256), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )
        self.reg_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feat_dim, 256), nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 4),
            nn.Sigmoid() # output in (0,1)
        )

    def forward(self, x):
        f = self.features(x)
        return self.cls_head(f), self.reg_head(f)
```

Illustrative Results:

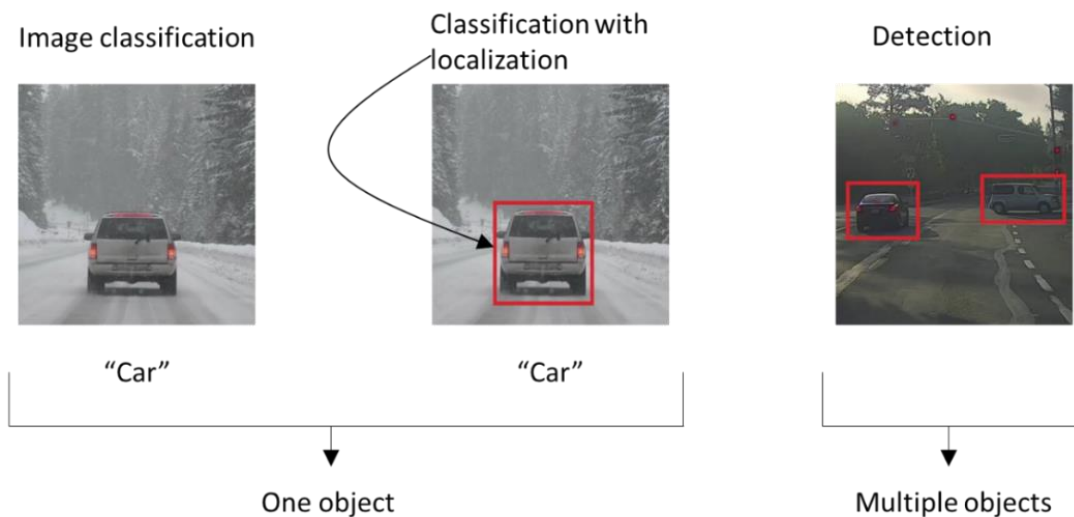


II. Object Detection

Faster RCNN

Faster R-CNN addresses the **object detection** problem, which differs from localization in that a single image may contain multiple objects. Each object requires its own bounding box and label.

The number of objects in each image is not fixed, so the output cannot be a fixed-size vector like in localization.

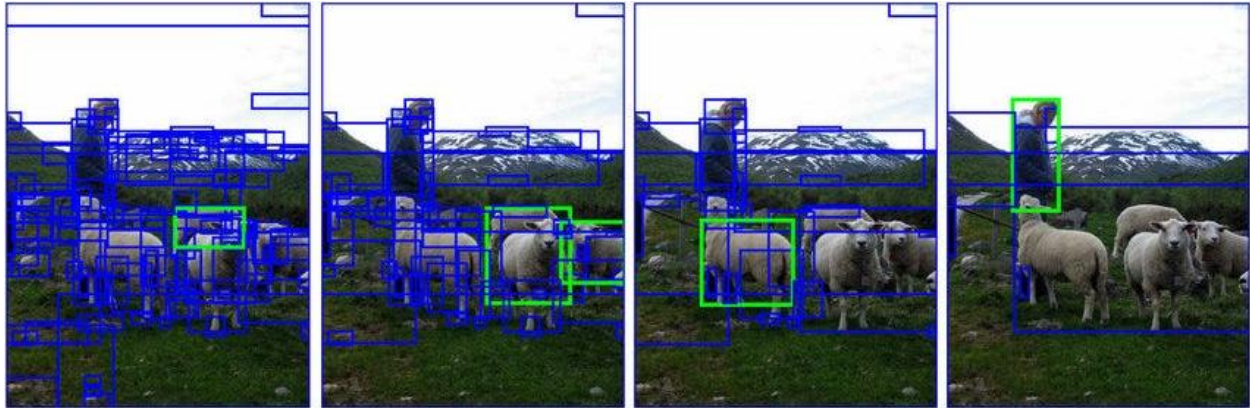


Before Faster R-CNN, the detection pipeline consisted of two separate steps:

1. Use traditional algorithms such as *Selective Search* to generate around ~2000 region proposals (possible object regions).
2. Feed each region into a CNN for classification.

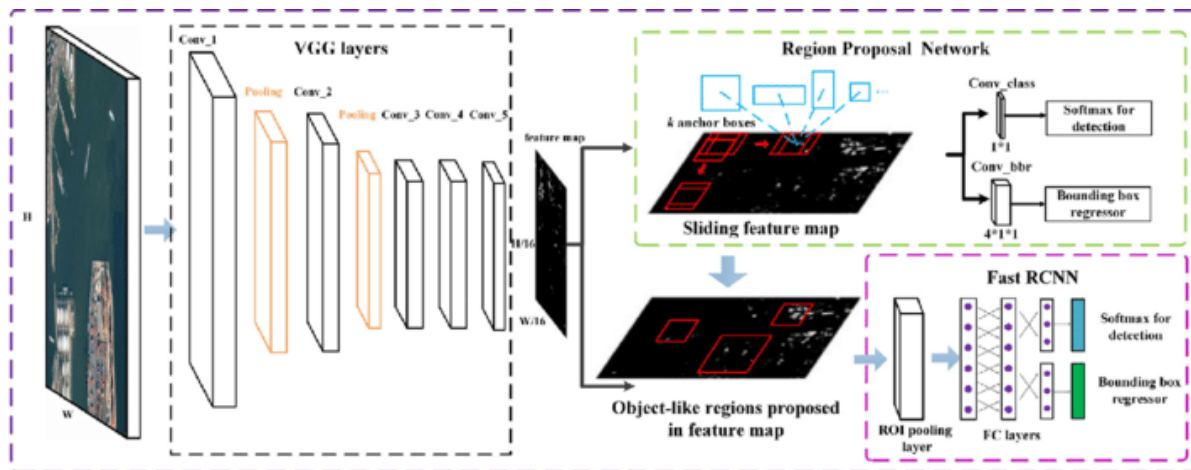
The main issues were:

- *Selective Search* runs on CPU and takes about ~2 seconds per image, becoming the bottleneck of the entire pipeline.
- The two steps are completely independent, meaning the CNN cannot influence the quality of the generated proposals.

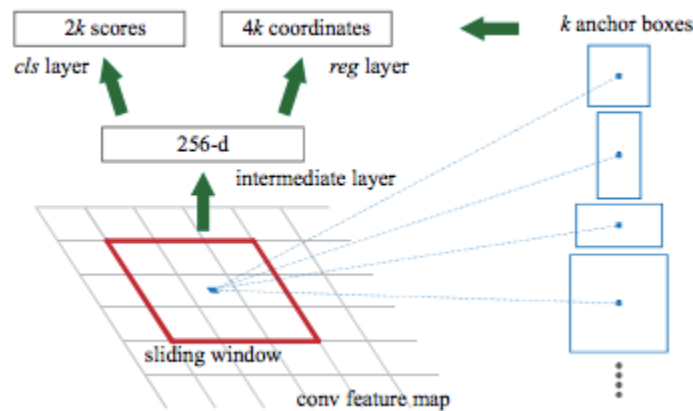


How Faster R-CNN Solves It

The core idea is to replace Selective Search with a small neural network operating directly on the backbone's feature map, called the **Region Proposal Network (RPN)**. Because the RPN shares features with the detection head, the entire pipeline can be trained **end-to-end**.



The RPN slides a small window over the feature map. At each location, it predicts k anchor boxes with different scales and aspect ratios. For each anchor, the RPN answers two questions: Is this region likely to contain an object? (objectness score) How should the box be adjusted to better fit the object? (box regression)

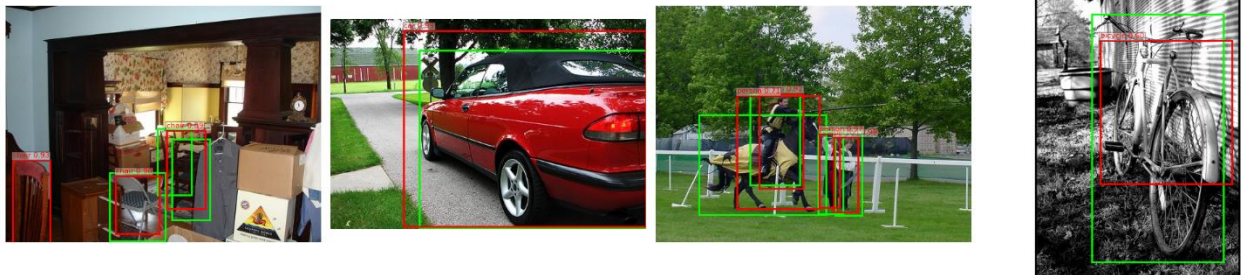


The proposals generated by the RPN are then: Normalized to a fixed size using RoI Pooling Passed to the detection head, which also consists of: A classification head (predict object class) A regression head (refine bounding box). Unlike localization, this process is applied independently to each proposal.

Implementation Code: (File *lab-4-faster-rcnn.ipynb*)

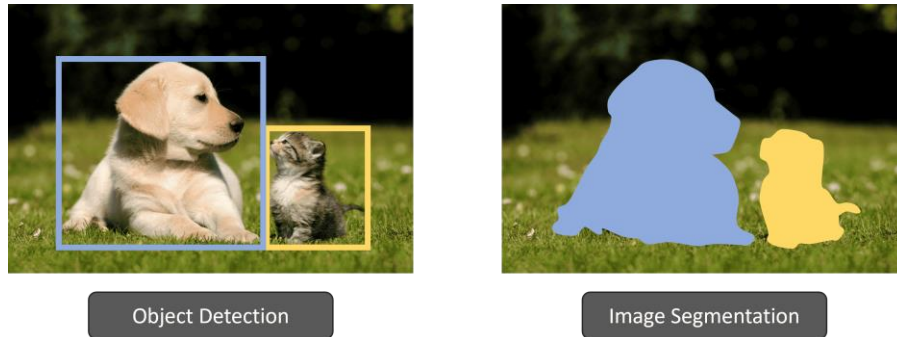
```
def get_model():
    model = fasterrcnn_resnet50_fpn(weights="DEFAULT")
    in_f = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_f, NUM_CLASSES)
    return model
```

Illustrative Results



III. Segmentation

The segmentation task requires assigning a label to every pixel in an image. The output is a mask with the same spatial dimensions ($H \times W$) as the input image.



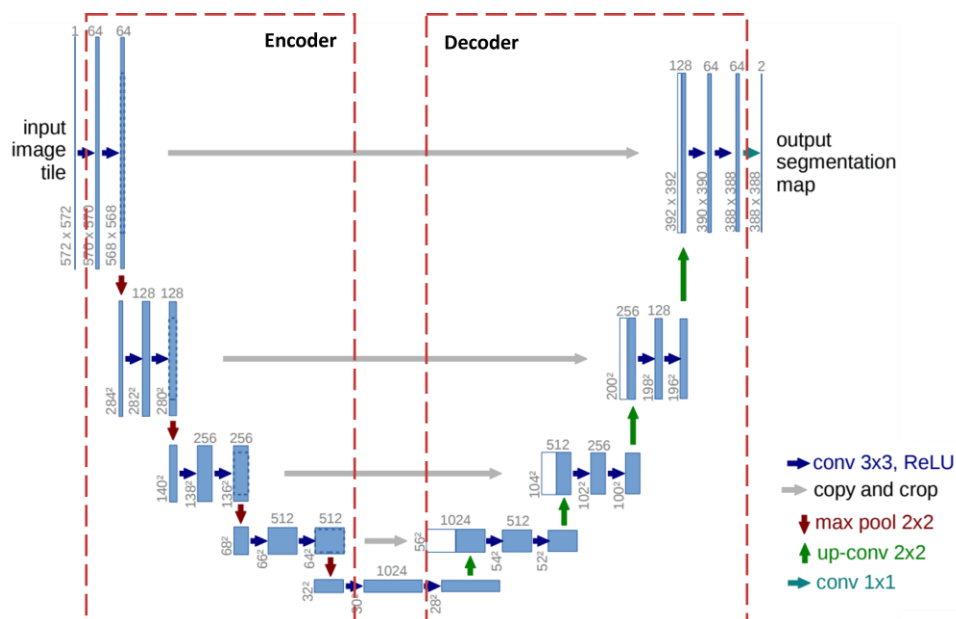
This constraint makes standard CNN encoders unsuitable for direct use, as they reduce the spatial resolution too much without a mechanism to recover it.

U-Net

Encoder

The **encoder** takes the input image and processes it through successive Conv and MaxPool blocks, gradually reducing H and W while increasing the number of channels.

- Shallow layers retain **low-level features** (edges, textures).
- Deeper layers capture **high-level features** (shapes, object types).
- However, precise spatial (pixel-level) information is progressively lost.



Bottleneck

The bottleneck is the deepest part of the network, where the feature map has the smallest spatial size and the largest channel depth. This is where the model captures the **global context** of the image before reconstructing the segmentation mask.

Decoder

The decoder progressively upsamples the feature maps to restore the original spatial resolution. However, simple upsampling is not sufficient because boundary information is lost during pooling. This is addressed by skip connections, which concatenate feature maps from the encoder (before pooling, where spatial details are preserved) with decoder features at the same resolution.

This helps the model recover fine-grained details and precise object boundaries.

Output

The final output is produced by a **1×1 convolution** that maps features to the number of classes.

- A **sigmoid** (for binary segmentation) or **softmax** (for multi-class segmentation) is applied to obtain probabilities.
- A threshold is then used to generate the final binary or categorical mask.

Each pixel in the output mask is the result of the entire pipeline:

- The encoder understands the content
- The bottleneck captures global context
- The decoder and skip connections reconstruct precise pixel-level boundaries

Convolutional Block Used

```
class DoubleConv(nn.Module):
    """(Conv2d → BN → ReLU) × 2"""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x): return self.block(x)
```

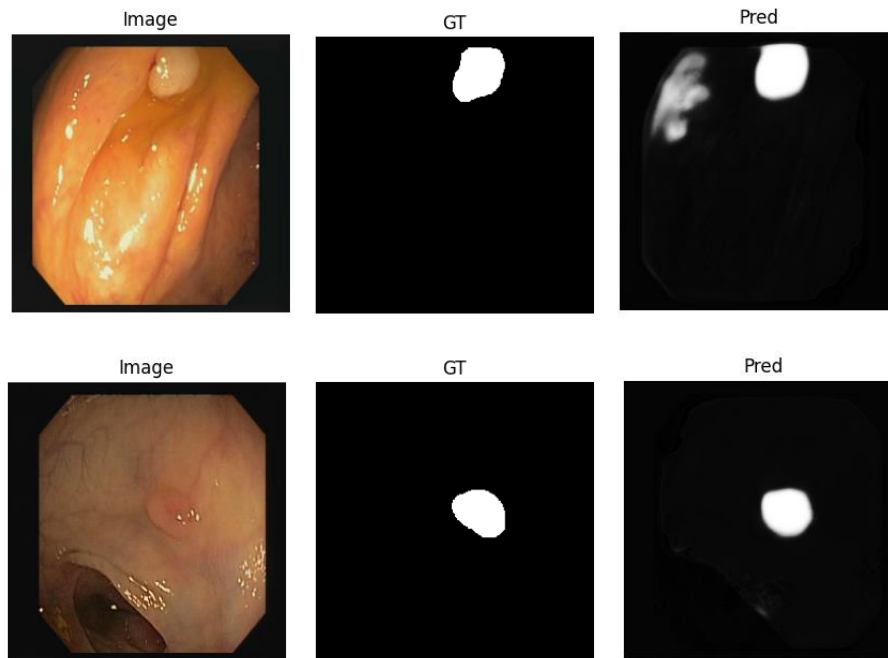

U-Net Architecture Implementation: (File *lab-4-unet.ipynb*)

```
class UNet(nn.Module):
    def __init__(self, in_ch=3, out_ch=1, features=(64,128,256,512)):
        super().__init__()
        # Encoder
        self.enc = nn.ModuleList()
        self.pool = nn.MaxPool2d(2, 2)
        ch = in_ch
        for f in features:
            self.enc.append(DoubleConv(ch, f))
            ch = f
        # Bottleneck
        self.bottleneck = DoubleConv(ch, ch*2)
        ch = ch*2
        # Decoder
        self.dec_up = nn.ModuleList()
        self.dec_conv = nn.ModuleList()
        for f in reversed(features):
            self.dec_up.append(
                nn.ConvTranspose2d(ch, f, kernel_size=2, stride=2)
            )
            self.dec_conv.append(DoubleConv(f*2, f)) # f*2 because of skip concat
            ch = f
        # Head
        self.head = nn.Conv2d(ch, out_ch, 1)

    def forward(self, x):
        skips = []
        for enc in self.enc:
            x = enc(x)
            skips.append(x)
            x = self.pool(x)
        x = self.bottleneck(x)
        skips = skips[::-1]
        for i, (up, conv) in enumerate(zip(self.dec_up, self.dec_conv)):
            x = up(x)
            s = skips[i]
            if x.shape != s.shape: # pad if needed
                x = TF.pad(x, [0, s.shape[3]-x.shape[3],
                               0, s.shape[2]-x.shape[2]])
            x = torch.cat([s, x], dim=1)
            x = conv(x)
        return self.head(x)
```

Link dataset: <https://www.kaggle.com/datasets/balraj98/cvcclinicdb>

Illustrative Results:



III. Contrastive Language-Image Pre-Training

CLIP is a **multimodal learning model** developed by OpenAI, designed to build a shared representation space for both images and text.

Instead of processing a single data type like traditional models, CLIP can understand and relate information across two different domains. This enables it to solve a wide range of tasks that combine **computer vision** and **natural language processing**.

Idea

The core idea behind CLIP is to leverage the massive amount of data available on the internet in the form of image–text pairs.

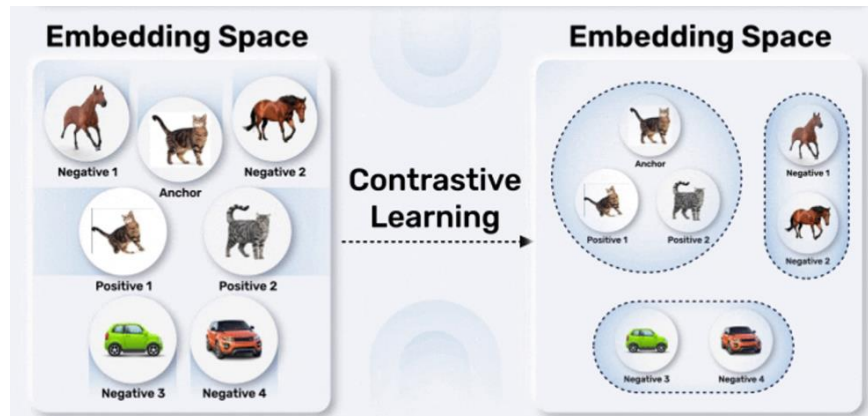
Rather than relying on manually labeled datasets, the model learns directly from natural language descriptions to understand image content. By learning to associate images with correct descriptions and distinguish them from incorrect ones, CLIP develops a high-level semantic understanding—closer to how humans describe the world.

Mechanism

CLIP is based on **contrastive learning**, a representation learning approach where the model learns to distinguish between:

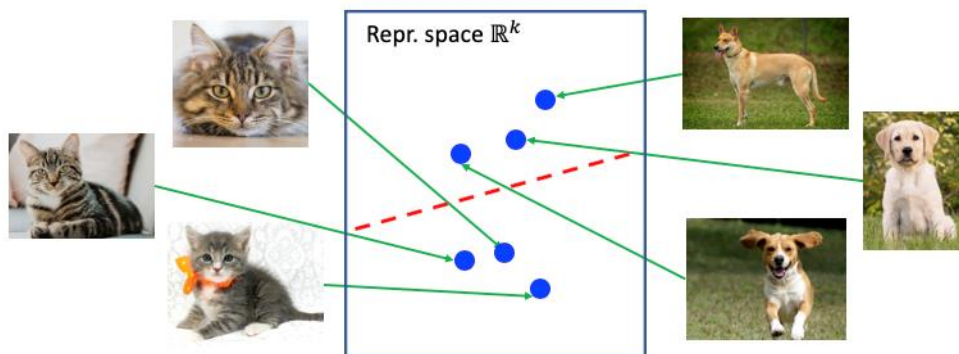
- **Positive pairs** (matching image–text pairs)

- **Negative pairs** (non-matching pairs)



Instead of learning from explicit class labels as in supervised learning, this method focuses on:

- Pulling related data representations closer in the vector space
- Pushing unrelated representations farther apart



Model Architecture

During training:

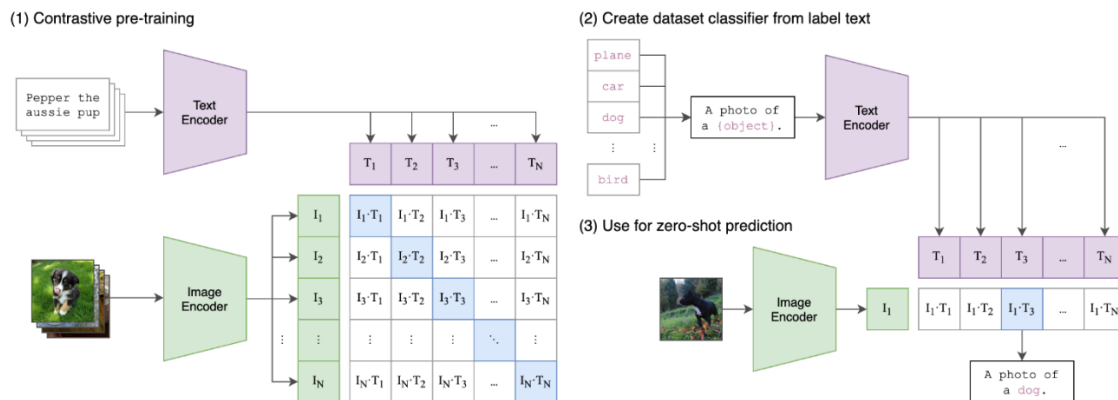
- Each image is paired with its correct description
- It is also compared against many incorrect descriptions within the same batch

The model maximizes similarity for correct pairs and minimizes similarity for incorrect ones. As a result, CLIP learns to map images and text into a **shared embedding space**, where related pairs are positioned closer together.

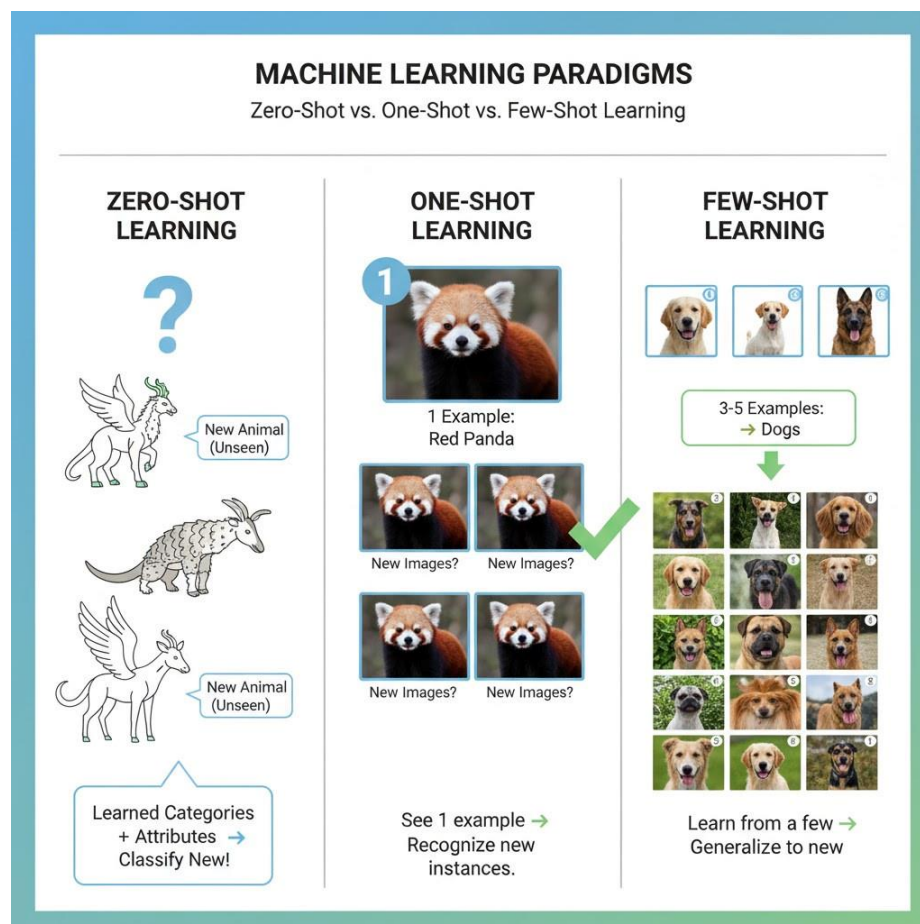
CLIP consists of two main components:

- **Image encoder:** Uses architectures such as ResNet or Vision Transformer to extract visual features
- **Text encoder:** Uses a Transformer to process text sequences

Both outputs are projected into the same embedding space and normalized before computing similarity using **cosine similarity**.



Low-shot learning:



1. Zero-shot

Zero-shot refers to the ability of a model to make predictions on classes it has never been explicitly trained on.

Example with CLIP:

- The model is not trained on the class “zebra”
- But it can still recognize it using a description like: “*a photo of a zebra*”

No training samples are required for each class.

2. One-shot

One-shot learning means the model only needs **one example** to learn a new class.

Example: Given only one image of a person → the model can recognize that person again

Requires **exactly one sample** per class.

3. Few-shot

Few-shot learning refers to learning from a small number of samples (typically 2–10 per class).

Example: Given 5 images of cats → the model learns to distinguish cats from other classes

Requires a **small number** (k) of samples per class.